Name:Samruddhi Sangram Yadav

Roll No.:560

Title: Implementing Feedforward neural networks with Keras and TensorFlow

a. Import the necessary packages

b. Load the training and testing data (MNIST/CIFAR10)

c. Define the network architecture using Keras

d. Train the model using SGD

e. Evaluate the network

f. Plot the training loss and accuracy

Code:

```
# a. Import the necessary packages
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.optimizers import SGD
# b. Load the MNIST training and testing data
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
print("Training data shape:", x_train.shape)
print("Testing data shape:", x_test.shape)
# Normalize pixel values between 0 and 1
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
# c. Define the network architecture using Keras
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation="relu"),
    Dense(64, activation="relu"),
    Dense(10, activation="softmax")
```

```
])
# Display the network architecture
model.summary()
# d. Compile and train the model using SGD
model.compile(
    optimizer=SGD(learning_rate=0.01),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.1,
    verbose=1
)
# e. Evaluate the network
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=0
)
print("\nTest Loss:", test_loss)
print("Test Accuracy: {:.2f}%".format(test_accuracy * 100))
# f. Plot the training loss and accuracy
# Plot training and validation loss
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
```

```
plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.title("Training and Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
# Plot training and validation accuracy
plt.subplot(1, 2, 2)
plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")
plt.title("Training and Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
# Predict a sample image
predictions = model.predict(x_test[:5], verbose=0)
print("\nSample Predictions:")
for i in range(5):
    predicted_digit = predictions[i].argmax()
    actual_digit = y_test[i]
    print(
        "Image", i + 1,
        "| Predicted Digit:", predicted_digit,
        "| Actual Digit:", actual_digit
    )
```

Output:

%runfile C:/Users/CD/.spyder-py3/test.py --wdir

Training data shape: (60000, 28, 28)

Testing data shape: (10000, 28, 28)

build TensorFlow with the appropriate compiler flags.

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386 (427.29 KB)

Trainable params: 109,386 (427.29 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8309 - loss: 0.6544 - val_accuracy: 0.9260 - val_loss: 0.2802

Epoch 2/10

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9126 - loss: 0.3080 - val_accuracy: 0.9383 - val_loss: 0.2250

Epoch 3/10

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9271 - loss: 0.2575 - val_accuracy: 0.9462 - val_loss: 0.1927

Epoch 4/10

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9357 - loss: 0.2238 - val_accuracy: 0.9532 - val_loss: 0.1694

Epoch 5/10

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9439 - loss: 0.1978 - val_accuracy: 0.9592 - val_loss: 0.1522

Epoch 6/10

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9497 - loss: 0.1768 - val_accuracy: 0.9625 - val_loss: 0.1386

Epoch 7/10

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.9546 - loss: 0.1597 - val_accuracy: 0.9638 - val_loss: 0.1282

Epoch 8/10

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.9588 - loss: 0.1453 - val_accuracy: 0.9663 - val_loss: 0.1199

Epoch 9/10

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9624 - loss: 0.1330 - val_accuracy: 0.9673 - val_loss: 0.1130

Epoch 10/10

1688/1688 ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.9651 - loss: 0.1223 - val_accuracy: 0.9693 - val_loss: 0.1075

Test Loss: 0.12285193055868149

Test Accuracy: 96.43%

Important

Figures are displayed in the Plots pane by default. To make them also appear inline in the console, you need to uncheck "Mute inline plotting" under the options menu of Plots.

Sample Predictions:

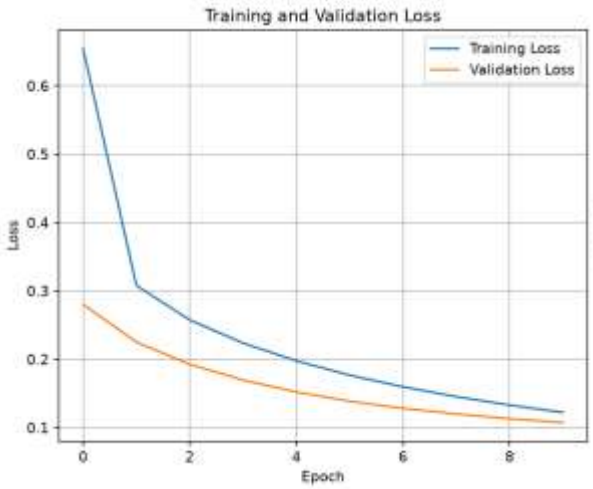
Image 1 | Predicted Digit: 7 | Actual Digit: 7

Image 2 | Predicted Digit: 2 | Actual Digit: 2

Image 3 | Predicted Digit: 1 | Actual Digit: 1

Image 4 | Predicted Digit: 0 | Actual Digit: 0

Image 5 | Predicted Digit: 4 | Actual Digit: 4


Training and Validation Loss
Training Loss
Validation Loss
Loss
Epoch
0.6
0.5
0.4
0.3
0.2
0.1
0
2
4
6
8


Training and Validation Accuracy
Training Accuracy
Validation Accuracy
Accuracy
Epoch
0.96
0.94
0.92
0.90
0.88
0.86
0.84
0
2
4
6
8